

Deep Learning Notes

Basic Training Loop in PyTorch — Hinglish mein, step by step

Is notes mein kya hai?

1. Abhi jo seekh rahe hain woh real world mein kahan kaam aata hai
2. Real-life problem (shop ki sales prediction) + data
3. Poora code aur har line ka matlab
4. Loss, gradient, update ki calculation (numbers ke saath)
5. 25 important terms: meaning + kaam + code + real life
6. Poora flow, exam answer, aur aage ka roadmap

1. Ye sab real world mein kahan kaam aayega?

Doubt valid hai: "Hum w, b, gradients mein phase hain, jabki Deep Learning se toh image aur text AI bante hain." Sach ye hai ki jo hum seekh rahe hain woh real Deep Learning ki **foundation** hai. Lekin sirf itne se ChatGPT ya image generator nahi banega, uske liye aage aur topics seekhne honge.

Stage	Kya banega	Real example	Concepts
1. Prediction (abhi yahan)	Data se relationship seekhna	House price, sales forecast, temperature	weights, bias, loss, gradients, training loop
2. Image recognition	Handwritten digit pehchanna (0-9)	Digit recognition, product classification	tensors, neural networks, nn.Module, CNN
3. Image generation	Text prompt se image	Night-time coding setup, landscape	deep networks, diffusion models, pretrained models
4. Text AI / LLMs	Chatbot, summary, Q&A, coding assistant	ChatGPT jaise tools	embeddings, attention, Transformers, LLMs

Jo seekha uska fayda

- Model prediction kaise karta hai.
- Loss se prediction ki galti kaise measure hoti hai.
- Gradients se parameters kaise update hote hain.
- Training ke saath loss kaise kam hota hai.

Ye concepts image classification aur text models ki training mein bhi use hote hain, bas wahan model aur process kaafi complex hota hai.

Yaad rakh: Deep Learning seekhne ka goal sirf formulas samajhna nahi, balki un formulas se **working models banana** hai. Isliye gradients par 10 aur examples practice karne ki zarurat nahi; optimizer + nn.Module samajhkar seedha MNIST project par jaayenge.

2. Real-life problem: shop ki sales prediction

Maan le teri shop hai. Tu advertising par kitna kharcha karta hai aur uske baad kitni sales hoti hain, iska data collect karta hai. Model purane data se seekhega aur naye advertising budget par sales predict karega. (Ye practice data hai, real measured data nahi.)

Advertising input (x)	1	2	3	4	5	6	7	8	9	10
Actual sales (y)	4	9	15	17	20	22	24	29	33	44

Poora code (pehle ek baar dekh)

```
import torch

# 1. Training data
x = torch.tensor([1., 2., 3., 4., 5., 6., 7., 8., 9., 10.])
y = torch.tensor([4., 9., 15., 17., 20., 22., 24., 29., 33., 44.])

# 2. Model parameters
w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

# 3. Learning rate
learning_rate = 0.01

# 4. Training loop
for epoch in range(1000):

    y_pred = w * x + b                                # Prediction
    loss = ((y_pred - y) ** 2).mean()                 # Loss (MSE)
    loss.backward()                                    # Gradients

    with torch.no_grad():                              # Update parameters
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad

    w.grad.zero_()                                     # Clear old gradients
    b.grad.zero_()

    if epoch % 100 == 0:
        print("Epoch:", epoch, "| Loss:", loss.item(),
              "| Weight:", w.item(), "| Bias:", b.item())

# 5. New prediction
new_x = torch.tensor(11.)
prediction = w * new_x + b
print("Predicted sales:", prediction.item())
```

3. Code ko line-by-line samajh

3.1 Data: x aur y

- **x** = model ko diye gaye inputs (advertising).
- **y** = un inputs ke actual answers (sales).
- Training mein model apni prediction ko actual answer se compare karta hai.
- Number ke baad . (jaise 1.) ka matlab floating-point number (1.0). Neural network calculations mein yehi common hai.

3.2 w aur b

```
w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
```

Model ka formula: **y_pred = w * x + b**

- **w** (weight): input aur output ke relationship ko control karta hai.
- **b** (bias): output ko adjust karta hai.
- **requires_grad=True**: PyTorch in parameters ke gradients calculate kar sakta hai.
- Starting mein dono 0 hain, yaani model ko abhi sahi relationship nahi pata.

3.3 Learning rate

learning_rate = 0.01 decide karta hai ki har update mein parameters kitne badhenge. *Analogy*: galti sudharte waqt chhota step lega toh slow hoga; bahut bada step lega toh sahi value se aage nikal sakta hai. 0.01 ek chosen setting hai, universal best value nahi.

3.4 for epoch in range(1000)

- Loop ke andar ka code 1,000 baar chalta hai. `range(1000)` = 0 se 999.
- **epoch** = current iteration ka number.
- Har epoch mein: prediction, loss, gradients, update. Yahan har epoch mein poore 10 data points use hote hain.

3.5 Prediction

y_pred = w * x + b model ka forward calculation hai. Starting mein $w=0$, $b=0$ isliye har prediction 0 hogi. Training mein w , b badalte hain, toh agle epoch mein predictions bhi badalti hain.

3.6 Loss: model ki galti ka number

```
loss = ((y_pred - y) ** 2).mean()
```

- **y_pred - y**: predicted aur actual ka difference.
- **** 2**: square karna, taaki positive aur negative errors cancel na hon.
- **.mean()**: saare squared errors ka average.

Isko **Mean Squared Error (MSE)** kehte hain: $MSE = (1/n) \times \text{sum of } (y_pred - y)^2$

Starting loss: har prediction 0 hai, toh actual values ke squares ka sum = $16+81+225+289+400+484+576+841+1089+1936 = 5937$. $n = 10$, isliye Loss = $5937 / 10 = 593.7$.
Dhyaan: loss kam = behtar (same data aur same loss function ke liye). Lekin training loss kam hona akela ye prove nahi karta ki model naye data par bhi achha chalega.

3.7 Gradient: loss.backward()

Ye line PyTorch ko calculate karne deti hai ki w aur b ko badalne se loss kis direction mein badhega ya ghathega.

- Starting par is data ke liye: **w.grad = -300.8** aur **b.grad = -43.4**.
- Negative gradient = in parameters ko chhota positive step dene se is point par loss kam hoga.
- **Important:** loss.backward() sirf gradients nikalta hai, w aur b ko khud update **nahi** karta.

3.8 Parameters update

```
with torch.no_grad():
    w -= learning_rate * w.grad
    b -= learning_rate * b.grad
```

torch.no_grad(): hum parameters ko manually modify kar rahe hain, toh PyTorch ko is update ko naya gradient-tracked calculation banane ki zarurat nahi.

Formula: **w_new = w_old - learning_rate x gradient**

Parameter	Calculation	Result
w	$0 - 0.01 \times (-300.8)$	3.008
b	$0 - 0.01 \times (-43.4)$	0.434

Ab model: **y_pred = 3.008x + 0.434**. $x = 1$ par prediction approx 3.442 (actual 4). Pehle ke 0 se behtar, par abhi exact nahi.

3.9 Gradients reset kyun?

```
w.grad.zero_()
b.grad.zero_()
```

PyTorch mein gradients by default **accumulate** hote hain. Reset nahi kiya toh naye gradient purane ke saath add ho jayenge. Hum chahte hain har iteration ka gradient sirf current loss se nikle, isliye update ke baad zero kar dete hain.

3.10 Output kaise padhein

- **epoch % 100 == 0:** sirf har 100 epochs par print.
- **.item():** single-value tensor ko normal Python number banata hai.

Epoch	Loss (approx.)	Weight	Bias
0	593.700	3.008	0.434
100	5.968	3.806	0.706
500	5.950	3.771	0.945
900	5.949	3.765	0.990

Loss bahut kam hua phir stabilize ho gaya: model ne parameters ko aisi values ki taraf move kiya jo training data par squared error kam karti hain.

Chhoti technical detail: epoch 0 par print hone wala loss update se *pehle* calculate hua, jabki printed weight/bias update ke *baad* ke hain. Isliye us row mein loss aur parameters bilkul ek hi model state ko represent nahi karte.

3.11 Naye input par prediction

```
new_x = torch.tensor(11.)  
prediction = w * new_x + b
```

Agar w approx 3.765 aur b approx 0.990 hon: $y_{\text{pred}} = 3.765 \times 11 + 0.990 = \text{approx } 42.4 \text{ units}$. Ye prediction hai, guarantee nahi. Alag test dataset use nahi kiya, isliye isse real-world accuracy establish nahi hoti.

4. Poora training loop ek flow mein

Step	Kya hota hai	Code
1. Training data	Inputs x aur actual answers y	x, y
2. Prediction	Model apna answer nikalta hai	y_pred = w*x + b
3. Loss	Prediction kitni wrong hai?	((y_pred - y)**2).mean()
4. Gradient	Parameters kis direction mein badlein?	loss.backward()
5. Update	w aur b adjust karo	w -= lr * w.grad
6. Clear	Purane gradients saaf	w.grad.zero_()
7. Repeat	Agla epoch: nayi prediction, naya loss	for epoch in range(...)

Exam-oriented answer (English)

A training loop is a repeated process used to train a machine learning model. In each epoch, the model generates predictions, calculates a loss, computes gradients using backpropagation, and updates its parameters to reduce the loss. The gradients are then cleared before the next iteration. After training, the learned parameters can be used to make predictions on new inputs. Model performance should also be evaluated on separate validation or test data.

5. Important terms: meaning + kaam + real life

A. Basic terms

1. Dataset	
Meaning	Data ka collection.
Kaam	Model ko examples dena jinse woh relationship seekhe.
Real life	Shop ke pichhle 10 din ki advertising aur sales ka record.
2. Feature / Input (x)	
Meaning	Woh information jo model ko prediction ke liye di jaati hai.
Kaam	Batana ki prediction kis information par based ho.
Real life	Advertising budget, house ka area, temperature, image ke pixel values.
3. Label / Target (y)	
Meaning	Woh actual answer jo predict karna seekhna hai.
Kaam	Model ki prediction ko correct answer se compare karna.
Real life	Actual sales, house price, image mein likha digit.
4. Model	
Meaning	Ek mathematical system jo input se output predict karta hai.
Kaam	Patterns seekhna aur naye inputs par predictions banana.
Real life	Hamara model $y_pred = w*x + b$, ek simple linear model hai.

B. Model ke andar ke terms

5. Weight (w)	
Meaning	Learnable parameter jo input ke effect ko control karta hai.
Kaam	Training mein adjust hota hai taaki error kam ho.
Real life	Advertising badhne par sales kitni badalti hain, ye represent karne wala coefficient.
Note	Weight ko real-world causation ka proof mat samajhna.

6. Bias (b)	
Meaning	Model ka additional learnable parameter.
Kaam	Output ko adjust karna, taaki model sirf zero se guzarti line tak limited na rahe.
Real life	Formula mein ek fixed adjustment add karne jaisa.

7. Parameter	
Meaning	Model ke andar woh value jo training se learn hoti hai.
Kaam	Model ke behaviour ko control karna.
Real life	Hamare model mein w aur b. Bade networks mein bahut saare parameters hote hain.

8. Prediction (y_pred)	
Meaning	Model ka nikala hua answer.
Kaam	Input ke liye estimated output dena.
Real life	Model ke hisaab se kal ki expected sales.

C. Training aur learning ke terms

9. Training	
Meaning	Data use karke model ke parameters learn karwana.
Kaam	Training examples par predictions improve karna.
Real life	Shop ke purane records par model ko baar-baar train karna.

10. Epoch	
Meaning	Training dataset ka ek complete pass.
Kaam	Batana ki model ne data kitni baar process kiya.
Real life	for epoch in range(1000): yahan 1,000 baar, har baar saare 10 points.

11. Loss	
Meaning	Prediction error ko measure karne wala number.
Kaam	Training mein signal dena ki predictions kitni galat hain.
Real life	Actual aur predicted sales ke differences ki average squared value (MSE).
Note	Loss kam hona training data par improvement dikhata hai, unseen data ki performance nahi.

12. Gradient	
Meaning	Batata hai ki parameter mein chhota change karne se loss kis direction mein aur kitni tezi se badlega.
Kaam	Parameters update karne ke liye direction aur magnitude dena.
Real life	Pahaad se neeche jaane ka rasta dhoondhna: gradient batata hai loss kis taraf badhta hai, hum uske opposite chalte hain.

13. Backpropagation	
Meaning	Neural network mein loss ke gradients ko chain rule se nikalne ka process.
Kaam	Har learnable parameter ke liye gradient nikalna.
Real life	Code: <code>loss.backward()</code> . Simple formula mein seedha calculation; bade networks mein layers ke through.

14. Learning rate	
Meaning	Update ka step size control karne wali setting.
Kaam	Decide karna ki har update mein parameters kitne badlenge.
Real life	Bahut bada = unstable loss; bahut chhota = slow learning.

15. Gradient descent	
Meaning	Gradients ke opposite direction mein parameters update karne ka method.
Kaam	Loss reduce karne wali parameter values dhoondhna.
Real life	Code: <code>w -= learning_rate * w.grad</code>

D. PyTorch ke special terms

16. Tensor	
Meaning	PyTorch ka numerical data structure (scalar, vector, matrix, higher-dimensional arrays).
Kaam	Data aur parameters store karna aur math operations karna.
Real life	<code>x = torch.tensor([1., 2., 3.])</code> ek 1-D tensor hai.

17. <code>requires_grad=True</code>	
Meaning	Tensor ke operations ke gradients track karne ki request.
Kaam	PyTorch ko batana ki is parameter ka gradient nikalna hai.

18. <code>loss.backward()</code>	
Meaning	Loss se gradients calculate karne ka command.
Kaam	<code>w.grad</code> , <code>b.grad</code> jaisi values populate karna.
Note	Ye parameters ko khud update nahi karta.

19. <code>torch.no_grad()</code>	
Meaning	Is block ke liye autograd tracking band karna.
Kaam	Manual parameter updates ko gradient-tracked computation se bahar rakhna.

20. .grad aur .zero_()	
Meaning	w.grad = stored gradient; .zero_() tensor ko in-place zero karta hai.
Kaam	Agli iteration se pehle purane gradients clear karna.
Note	PyTorch gradients accumulate karta hai, isliye reset zaroori hai.

21. .item()	
Meaning	Single-value tensor se normal Python number nikalna.
Kaam	Loss, weight, bias ko readable format mein print karna.
Real life	print(loss.item())

E. Training ke baad wale terms

22. Optimizer	
Meaning	Gradients ke basis par parameters update karne wala algorithm.
Kaam	Manual update code ki jagah updates manage karna.
Real life	optimizer = torch.optim.SGD([w, b], lr=0.01)
Note	optimizer.step() = update, optimizer.zero_grad() = gradients clear. Yahi next topic hai.

23. Train / Validation / Test data	
Meaning	Data ko alag purposes ke liye divide karna.
Kaam	Training: parameters seekhne ke liye. Validation: settings/choices evaluate karne ke liye. Test: final evaluation, aise data par jo selection mein use nahi hua.
Real life	Purane records par train, alag records se settings choose, untouched records par final check.

24. Accuracy	
Meaning	Classification mein correct predictions ka proportion.
Kaam	Batana ki kitni predictions sahi hain.
Note	Regression (sales, house price) mein MSE, MAE jaise metrics use hote hain. Har task ke liye accuracy suitable nahi.

25. Generalization	
Meaning	Model ka naye, unseen data par bhi theek perform karna.
Kaam	Ye jaanchna ki model ne sirf yaad kiya ya useful patterns bhi seekhe.
Real life	Isliye test data par evaluation zaroori hai.

optimizer wale 3 commands (jo aage use honge)

Command	Matlab
loss.backward()	gradient calculate
optimizer.step()	weights/bias update
optimizer.zero_grad()	purane gradients clear

Main point: `loss.backward()` batata hai parameters ko kis direction mein adjust karna hai; optimizer (ya manual gradient descent) update karta hai; aur test data batata hai ki model naye examples par kitna achha kaam karta hai.

6. Aage ka roadmap

Step	Topic	Kya karenge
1	Basic training loop	Concept samajh liya (prediction, loss, gradients, update, epochs)
2	Optimizer + <code>nn.Module</code> + <code>nn.Linear</code>	Manual code ko proper PyTorch style model mein convert karenge
3	Neural Network (MLP)	Multiple neurons, layers, activation functions, forward pass
4	MNIST project	Handwritten digits par train karenge, test accuracy aur galat predictions dekhenge
5	CNN + image classification	Images ke spatial patterns seekhna, ek real image classifier banana
6	Transfer Learning, Transformers, GenAI	Pretrained models, text processing, LLMs, diffusion models

Next kya?

- Training loss kam karna aur genuinely unseen test data par performance check karna **alag** cheezein hain, ye samajhna.
- Manual update ki jagah **`torch.optim.SGD`** use karna.
- Phir isi loop ko **`nn.Module` / `nn.Linear`** wale actual neural network mein convert karna.
- Uske baad MNIST: tu image dega, model digit predict karega, aur hum dekhenge kitni predictions sahi hain.